

A photograph of a modern building with a curved glass facade. The word "BANK" is prominently displayed in large, white, three-dimensional letters on a tan-colored panel above the glass. The glass reflects the surrounding environment, including other buildings and the sky.

BANK

Banking Transaction & Account Management System

This project focuses on designing and analyzing a banking system that not only manages customer accounts and transactions but also generates meaningful insights from transaction data to support business decision-making.
(Using Python)

Presented By
Prakash Chawda



Problem Statement

Banks generate massive volumes of transaction data daily, but without proper analysis, it becomes difficult to monitor customer behavior, detect irregularities, and optimize financial services.

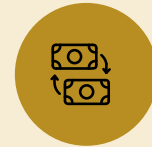
There is a critical need for a system that manages accounts efficiently while providing analytical insights for data-driven decision-making.

Project Objectives



Account Management

Develop Python-based banking system for customer accounts and transactions



Transaction Tracking

Monitor deposits, withdrawals, and transfers with validation

Methodology



Account Creation

Collect user details and generate unique account numbers



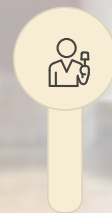
Transaction Module

Process deposits, withdrawals, and transfers with validation



Data Storage

Save and load data using pickle files at program start



Reporting System

Display transaction history and generate summary reports



Error Handling

Validate inputs and prevent invalid transactions



```
import pickle
import os
from datetime import datetime

FILE_NAME = "bank_data.pkl"

# ----- LOAD & SAVE -----
def load_data():
    if os.path.exists(FILE_NAME):
        with open(FILE_NAME, "rb") as f:
            return pickle.load(f)
    return {}

def save_data(data):
    with open(FILE_NAME, "wb") as f:
        pickle.dump(data, f)

# ----- ACCOUNT CLASS -----
class Account:
    def __init__(self, name, acc_type, balance, password):
        self.name = name
        self.acc_type = acc_type
        self.balance = balance
        self.password = password
        self.transactions = []

    def add_transaction(self, t_type, amount):
        self.transactions.append({
            "type": t_type,
            "amount": amount,
            "date": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        })
```

```
# ----- HELPER -----
def generate_account_number(data):
    return str(len(data) + 2008001)

def check_password(acc):
    p = input("Enter password: ")
    if p == acc.password:
        return True
    else:
        print("----Invalid password----")
        return False

# ----- FUNCTIONS -----
def create_account(data):
    name = input("Enter name: ")
    acc_type = input("Account type (Savings/Current): ")
    password = input("Set password: ")

    try:
        balance = float(input("Initial balance: "))
        if balance < 0:
            print("----Balance cannot be negative----")
            return
    except:
        print("----Invalid balance input----")
        return

    acc_no = generate_account_number(data)
    acc = Account(name, acc_type, balance, password)
    acc.add_transaction("Initial Deposit", balance)

    data[acc_no] = acc
    print(f"----Account created! Account No: {acc_no}")
```



```
def view_account(data):
    acc_no = input("Enter account number: ")
    acc = data.get(acc_no)

    if acc:
        if not check_password(acc):
            return

        print("\n--- Account Details ---")
        print(f"Name: {acc.name}")
        print(f>Type: {acc.acc_type}")
        print(f"Balance: ₹{acc.balance}")
    else:
        print("----Account not found----")
```

```
def deposit(data):
    acc_no = input("Enter account number: ")
    acc = data.get(acc_no)

    if acc:
        if not check_password(acc):
            return

        try:
            amount = float(input("Enter deposit amount: "))
            if amount <= 0:
                print("----Amount must be positive----")
                return

            acc.balance += amount
            acc.add_transaction("Deposit", amount)
            print("----Deposit successful----")

        except Exception as e:
            print("----Invalid input----", e)
    else:
        print("----Account not found----")
```

```
def withdraw(data):
    acc_no = input("Enter account number: ")
    acc = data.get(acc_no)

    if acc:
        if not check_password(acc):
            return

        try:
            amount = float(input("Enter withdrawal amount: "))

            if amount <= 0:
                print("----Amount must be positive----")
                return

            if amount > acc.balance:
                print("----Insufficient balance----")
                return

            acc.balance -= amount
            acc.add_transaction("Withdrawal", amount)
            print("----Withdrawal successful----")

        except Exception as e:
            print("----Invalid input----", e)
    else:
        print("----Account not found----")
```



```
def transfer(data):
    from_acc_no = input("From account: ")
    to_acc_no = input("To account: ")

    if from_acc_no not in data or to_acc_no not in data:
        print("----Account not found----")
        return

    from_acc = data[from_acc_no]
    to_acc = data[to_acc_no]

    if not check_password(from_acc):
        return

    try:
        amount = float(input("Enter amount: "))

        if amount <= 0:
            print("----Amount must be positive----")
            return

        if amount > from_acc.balance:
            print("----Insufficient balance----")
            return

        from_acc.balance -= amount
        to_acc.balance += amount

        from_acc.add_transaction("Transfer Sent", amount)
        to_acc.add_transaction("Transfer Received", amount)
```

```
        print("----Transfer successful----")

    except Exception as e:
        print("----Invalid input----", e)

def transaction_history(data):
    acc_no = input("Enter account number: ")
    acc = data.get(acc_no)

    if acc:
        if not check_password(acc):
            return

        print("\n" + "="*60)
        print(f"{'DATE & TIME': <20} {'TYPE': <20} {'AMOUNT' : <10}")
        print("="*60)

        for t in acc.transactions:
            print(f"{t['date']:<20} {t['type']:<20} ₹{t['amount']:>8.2f}")
            print("="*60)
    else:
        print("----Account not found----")
```



Banking System Interface



```
# ----- MENU -----
def menu():
    data = load_data()

    while True:
        print("\n===== BANK MENU =====")
        print("1. Create Account")
        print("2. View Account")
        print("3. Deposit")
        print("4. Withdraw")
        print("5. Transfer")
        print("6. Transaction History")
        print("7. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            create_account(data)
        elif choice == "2":
            view_account(data)
        elif choice == "3":
            deposit(data)
        elif choice == "4":
            withdraw(data)
        elif choice == "5":
            transfer(data)
        elif choice == "6":
            transaction_history(data)
        elif choice == "7":
            save_data(data)
            print("Data saved. Exiting...")
            #PRAKASH CHAWDA
            break
        else:
            print("----Invalid choice----")
```

```
# ----- RUN -----
if __name__ == "__main__":
    menu()
```


Sample Output :



```
===== BANK MENU =====
1. Create Account
2. View Account
3. Deposit
4. Withdraw
5. Transfer
6. Transaction History
7. Exit
Enter choice: 1
Enter name: Prakash
Account type (Savings/Current): Savings
Set password: 12345
Initial balance: 10000
----Account created! Account No: 2008001
```

```
===== BANK MENU =====
1. Create Account
2. View Account
3. Deposit
4. Withdraw
5. Transfer
6. Transaction History
7. Exit
Enter choice: 1
Enter name: Bhawani Singh
Account type (Savings/Current): Current
Set password: 6789
Initial balance: 15000
----Account created! Account No: 2008002
```

```
===== BANK MENU =====
1. Create Account
2. View Account
3. Deposit
4. Withdraw
5. Transfer
6. Transaction History
7. Exit
Enter choice: 2
Enter account number: 2008002
Enter password: 6789

--- Account Details ---
Name: Bhawani Singh
Type: Current
Balance: ₹15000.0
```

```
===== BANK MENU =====
1. Create Account
2. View Account
3. Deposit
4. Withdraw
5. Transfer
6. Transaction History
7. Exit
Enter choice: 3
Enter account number: 2008001
Enter password: 12345
Enter deposit amount: 5000
----Deposit successful----
```

```
===== BANK MENU =====
1. Create Account
2. View Account
3. Deposit
4. Withdraw
5. Transfer
6. Transaction History
7. Exit
Enter choice: 4
Enter account number: 2008002
Enter password: 6789
Enter withdrawal amount: 2000
----Withdrawal successful----
```

```
===== BANK MENU =====
1. Create Account
2. View Account
3. Deposit
4. Withdraw
5. Transfer
6. Transaction History
7. Exit
Enter choice: 5
From account: 2008001
To account: 2008002
Enter password: 12345
Enter amount: 4530.45
----Transfer successful----
```



Passbook Entry

===== BANK MENU =====

1. Create Account
2. View Account
3. Deposit
4. Withdraw
5. Transfer
6. Transaction History
7. Exit

Enter choice: 6

Enter account number: 2008002

Enter password: 6789

DATE & TIME	TYPE	AMOUNT
2026-04-16 15:23:58	Initial Deposit	₹15000.00
2026-04-16 15:27:40	Withdrawal	₹ 2000.00
2026-04-16 15:28:53	Transfer Received	₹ 4530.45

===== BANK MENU =====

1. Create Account
2. View Account
3. Deposit
4. Withdraw
5. Transfer
6. Transaction History
7. Exit

Enter choice: 6

Enter account number: 2008001

Enter password: 12345

DATE & TIME	TYPE	AMOUNT
2026-04-16 15:23:25	Initial Deposit	₹10000.00
2026-04-16 15:26:27	Deposit	₹ 5000.00
2026-04-16 15:28:53	Transfer Sent	₹ 4530.45

Accomplishments & Future Vision

What We Built

- Fully functional banking system
- Transaction tracking with validation
- Structured dataset for analysis
- Meaningful financial insights

Future Opportunities

- Database integration (MySQL/PostgreSQL)
- Visualization tools (Power BI/Tableau)
- ML-based fraud detection
- Real-time analytics dashboard
- API integration for live systems



[LinkedIn.com](https://www.linkedin.com)



[GitHub.com](https://github.com)

Conclusion

This project successfully combines banking operations with data analytics, demonstrating how transaction data can be used to derive meaningful insights.

It highlights the importance of data-driven decision-making in financial systems and provides a foundation for building more advanced banking analytics solutions.

Recommendations

- Advanced authentication (OTP, encryption)
- Graphical dashboards for insights
- Cloud storage for scalability
- AI-based anomaly detection

Limitations

- System-generated records only
- No advanced visualization
- Basic security features
- Limited scalability